SLE-4: ARCHITECTURE DECISION RECORD (ADR)
BFS and DFS Graph Search System

PRN: 26UAM307

Name: SHRAVANI SADASHIV BRAMHADANDE

Division: A

Date: 11/10/2026

Status: Accepted

PART A – ARCHITECTURE DECISION RECORD (ADR)

ADR-01: Keeping BFS and DFS as Separate Search Modules

1. Context

The project is a BFS and DFS Graph Search System. In SLE-2, I implemented and compared Breadth-First Search (BFS) and Depth-First Search (DFS) on a graph with start node A and goal node AD. The work recorded execution time, the number of nodes expanded, and the path found by each algorithm.

In SLE-3, I continued this work by designing a Full C4 architecture for a Traffic Signal Optimizer Agent. The architecture separates graph input, BFS search, DFS search, performance measurement/profiling, and output responsibilities. This raised an architectural question: should the Search Engine keep both algorithms as separate modules, or should one algorithm replace the other?

The purpose of this ADR is to record the decision, the alternatives considered, the evidence from SLE-2, and the consequences for the SLE-3 design.

2. Decision

I decided to keep BFS and DFS as separate search modules inside the Search Engine of the Traffic Signal Optimizer Agent. The modules will remain independently testable and maintainable. The SLE-2 timing results will be used as evidence for the tested graph only, not as a universal rule for every graph or traffic situation.

3. Alternatives Considered

Alternative 1: Use BFS only

- **Description**: Use Breadth-First Search as the single graph-search strategy.
- **Advantages**: BFS explores level by level and, for an unweighted graph, finds a path with the fewest edges when the usual BFS conditions are met.
- **Reason not selected**: BFS was faster in the recorded SLE-2 test, but that result alone does not prove BFS will be the best choice for every graph or future traffic-management task.

Alternative 2: Use DFS only

- **Description:** Use Depth-First Search as the single graph-search strategy.
- **Advantages**: DFS can explore a branch deeply and may be useful when depth-oriented exploration is appropriate.
- **Reason not selected**: DFS took more measured time than BFS in the reported SLE-2 experiment. Choosing DFS alone would also remove the ability to compare the two implementations within the same system.

Alternative 3: Keep BFS and DFS as separate modules (Chosen)

- **Description**: Maintain both algorithms as distinct modules that can be run on the same graph and goal.
- **Reason for selection**: This supports direct comparison, independent testing and maintenance, and future evaluation using different graph structures and conditions.

4. Rationale / Justification

The SLE-2 repository reports the following results for the graph search from A to AD:

Metric	BFS	DFS
Average execution time	0.011309 ms	0.013244 ms
Nodes expanded	30	30
Path found	A → C → G → O → AD	A → C → G → O → AD

- BFS was approximately 14.6% faster than DFS in this reported experiment.
- Both algorithms expanded 30 nodes and returned the same path.
- The measurements show a difference in recorded execution time, but they do not show that BFS is always faster or more suitable.

Keeping both modules is therefore a more balanced architectural decision than permanently removing one algorithm based on one graph test. It supports future testing with different graphs and makes it easier to replace, improve, or compare an algorithm without rewriting the other one.

This decision also fits the SLE-3 C4 design, which separates the graph input, search modules, performance measurement/profiling, and output. The SLE-2 numbers describe the selected test graph and testing environment; they are not measured results from a live traffic signal system.

5. Consequences

Positive Consequences

- BFS and DFS can be tested, debugged, and maintained independently.
- The same graph and goal can be used to compare algorithm behaviour under consistent conditions.
- The Search Engine remains modular, supporting future improvements and additional search strategies.
- Profiling can remain separate from the core search logic, making results easier to record and interpret.

Negative Consequences / Trade-offs

- Keeping two implementations requires additional code review, tests, and maintenance.
- If the system needs to select one algorithm automatically, a clear selection rule will need to be designed and tested.
- Timing results can vary with graph structure, implementation details, hardware, system load, and measurement method.
- The SLE-2 graph experiment is not enough to decide which algorithm is best for a real traffic-management scenario.

6. References

SLE-1 – AI Agent repository and agent.py: **https://github.com/shravani9103/AI-Agent-SLE1**

SLE-2 and SLE-3 – BFS/DFS implementation and architecture repository:

https://github.com/shravani9103/AI-SLE2

SLE-3 – Full C4 Architecture Design for the Traffic Signal Optimizer Agent (submitted project report).

PART B – SHORT SLE JOURNEY

SLE-1: AI Agent

In SLE-1, I worked on an AI agent project and shared the agent.py file in my GitHub repository. The code uses the Ollama Python package to send chat messages to the Llama 3.2 model and display responses. This phase helped me understand an agent's basic program flow, model interaction, user input, and debugging.

SLE-2: BFS and DFS Profiling

In SLE-2, I implemented and compared BFS and DFS on a graph from start node A to goal node AD. I recorded the execution time, number of nodes expanded, and path found. The repository reports average execution times of 0.011309 ms for BFS and 0.013244 ms for DFS. Both algorithms expanded 30 nodes and found A → C → G → O → AD.

SLE-3: Full C4 Architecture

In SLE-3, I designed the Full C4 architecture for the Traffic Signal Optimizer Agent using the BFS/DFS work as the search foundation. The four levels are Context, Container, Component, and Code. The design identifies graph input, BFS search, DFS search, performance measurement/profiling, and output responsibilities.

SLE-4: ADR and Viva

In SLE-4, I selected an architectural decision that connects SLE-2 profiling evidence with the SLE-3 design. This ADR records why BFS and DFS should remain separate modules, what alternatives were considered, which profiling evidence supports the decision, and the benefits and trade-offs. The viva notes summarize the main points to defend.

PART C – AI CONTRIBUTION NOTE

AI tools used: ChatGPT and GitHub Copilot, if applicable to the work completed.

What AI helped with: Explaining algorithm concepts, providing debugging guidance, helping interpret profiling measurements, organizing the ADR sections, and improving documentation clarity.

What I did myself: I reviewed the suggestions, examined the code and project results available to me, connected the SLE-2 profiling with the SLE-3 C4 architecture, and made the final design decisions. This statement should be edited if any part does not accurately describe my own contribution.

PART D – VIVA PREPARATION NOTES & DEFENSE GUIDE

1. Core Architectural Decision

I kept BFS and DFS as separate modules inside the Search Engine so they can be tested and maintained independently and compared on the same graph.

2. Empirical Evidence from SLE-2

The reported average execution time was 0.011309 ms for BFS and 0.013244 ms for DFS. Both expanded 30 nodes and found the same path, A → C → G → O → AD. BFS was faster in this particular experiment.

3. Why not use BFS only?

BFS was faster in the recorded test, but one graph and one testing environment do not prove it is always the best choice. Keeping both algorithms allows future comparisons and avoids making a universal claim from limited evidence.

4. Why not use DFS only?

DFS provides depth-oriented exploration, but it took longer in the recorded test. Removing BFS would also remove the ability to compare both search strategies directly.

5. What is an ADR?

An Architecture Decision Record is a document that explains an important design decision, its context, the alternatives considered, the reason for the selected option, and the consequences.

6. What are the four C4 levels?

Context shows the system and its external relationships. Container shows the major applications or deployable/structural units. Component shows the major parts within a container. Code shows implementation-level structures such as classes and functions.

7. What does profiling tell us?

Profiling and timing help compare observed program behaviour and locate potential performance bottlenecks. Results must be interpreted in the context of the graph, environment, and measurement method.

8. What is a limitation of my evidence?

The measurements come from the selected SLE-2 graph. They should not be presented as proof of performance on a real traffic network or as evidence that BFS always outperforms DFS.

CONCLUSION

The main architectural decision in this report is to keep BFS and DFS as separate search modules in the Traffic Signal Optimizer Agent. SLE-2 reported a lower average execution time for BFS than DFS, while both expanded 30 nodes and found the same path. This evidence supports comparison but does not establish a universal winner. The modular SLE-3 C4 architecture provides a suitable place for both search strategies, with profiling kept separate from search logic. This ADR documents the context, decision, alternatives, rationale, consequences, and relationship to the overall SLE journey.